



Paris 1 Panthéon-Sorbonne University
U.F.R. 27 – Mathematics and Computer Science

PROJECT REPORT
MASTER 2 MMMEF
MATHEMATICAL METHODS IN ECONOMICS AND FINANCE

TOPIC

Conjugate Gradient Method: Theory, Implementation and Applications

Written by:

- Marc SU

Supervised by:

- Professor Gisella Croce

Academic year 2025 – 2026

Acknowledgements

I would like to sincerely thank my professor for the convex analysis course and for the mathematical material that made this project possible. This work allowed me to deepen my understanding of iterative methods, quadratic optimization and numerical linear algebra.

I would also like to thank the teaching team for the guidance provided throughout the semester, which helped me connect theoretical results with practical implementation.

Executive Summary

Key idea of the project

This project studies the Conjugate Gradient method as both a numerical linear algebra algorithm and an optimization method. The central idea is to solve a symmetric positive definite linear system $Ax = b$ by minimizing the strictly convex quadratic function

$$f(x) = \frac{1}{2}x^\top Ax - b^\top x.$$

The report emphasizes the link between theory and implementation: the method is derived from exact line search, the search directions are built to be A -conjugate, and the iterates belong to Krylov subspaces. From a computational point of view, the algorithm is attractive because it avoids matrix inversion and only requires matrix-vector products, scalar products and vector updates.

The main contribution of the numerical part is to implement the method from scratch in Python, verify the assumptions on the matrix, compare the result with a direct solver, and track the residual norm along the iterations. This reinforces the interpretation of the method as a bridge between optimization theory, numerical linear algebra and scientific computing.

Introduction

The Conjugate Gradient method is a fundamental iterative algorithm in numerical linear algebra and optimization. It is mainly used to solve large linear systems of the form

$$Ax = b,$$

where A is symmetric positive definite. Such systems appear in many applications, including scientific computing, engineering, numerical optimization and discretized partial differential equations.

The purpose of this report is to present the theoretical foundations of the Conjugate Gradient method, its interpretation as an optimization algorithm, its finite convergence property, and its connection with Krylov subspaces. The report also includes a detailed numerical example that illustrates the construction of gradients, step lengths and conjugate directions.

The Conjugate Gradient method is remarkable because it combines the efficiency of iterative methods with a finite convergence guarantee in exact arithmetic. Unlike steepest descent, which

may converge slowly when the matrix is ill-conditioned, the Conjugate Gradient method builds search directions that are mutually conjugate with respect to the matrix A . This property prevents the algorithm from undoing progress made in previous iterations.

Contents

Acknowledgements	1
Executive Summary	1
Introduction	1
1 Mathematical Framework	5
1.1 The linear system	5
1.2 Inner product induced by a positive definite matrix	5
1.3 Equivalent quadratic minimization problem	6
1.4 Residual, error and gradient	7
2 Conjugate Gradient Method	9
2.1 A-conjugacy	9
2.2 Exact line search	9
2.3 Algorithm	10
2.4 Interpretation of the steps	11
3 Orthogonality and Finite Convergence	12
3.1 Orthogonality of gradients	12
3.2 Krylov subspaces	12
4 Detailed Numerical Example	14
4.1 Problem setup	14
4.2 Initialization	14
4.3 First iteration	14
4.4 Second and third iterations	15
4.5 Verification properties	15
4.6 Numerical verification with Python	15
5 Applications and Practical Considerations	16
5.1 Geometric interpretation	16
5.2 Preconditioning	16
5.3 Implementation strategy	16

5.4	Stopping criteria and numerical stability	17
5.5	Computational cost	17
5.6	Preconditioned Conjugate Gradient method	18
5.7	Practical workflow	18
5.8	Possible extensions	19
5.9	Advantages and limitations	19
Conclusion		20
References		21
Appendix: Python Code		22

Mathematical Framework

1.1 The linear system

We consider the linear system

$$Ax = b,$$

where

$$A \in \mathbb{R}^{n \times n}, \quad b \in \mathbb{R}^n, \quad x \in \mathbb{R}^n.$$

Definition 1.1 (Symmetric positive definite matrix). A matrix $A \in \mathbb{R}^{n \times n}$ is symmetric positive definite if

$$A^\top = A$$

and

$$x^\top Ax > 0$$

for every nonzero vector $x \in \mathbb{R}^n$.

Remark 1.1. The assumption that A is symmetric positive definite is essential for the classical Conjugate Gradient method. It guarantees that the associated quadratic function is strictly convex and has a unique minimizer.

1.2 Inner product induced by a positive definite matrix

A central idea in the Conjugate Gradient method is that orthogonality is not only measured with the usual Euclidean scalar product. Since the matrix A is symmetric positive definite, it defines its own geometry.

Definition 1.2 (A -inner product and A -norm). Let $A \in \mathbb{R}^{n \times n}$ be symmetric positive definite. For $x, y \in \mathbb{R}^n$, the A -inner product is defined by

$$\langle x, y \rangle_A = x^\top Ay.$$

The associated A -norm is

$$\|x\|_A = \sqrt{x^\top Ax}.$$

Proposition 1.1 (The A -inner product is well-defined). If A is symmetric positive definite, then $\langle x, y \rangle_A = x^\top Ay$ defines an inner product on $\mathbb{R}^n$.

Proof

We verify the three main properties of an inner product. First, for all $x, y \in \mathbb{R}^n$, bilinearity follows from the linearity of matrix multiplication. Second, since A is symmetric, we have

$$\langle x, y \rangle_A = x^\top A y = (x^\top A y)^\top = y^\top A^\top x = y^\top A x = \langle y, x \rangle_A.$$

Finally, since A is positive definite,

$$\langle x, x \rangle_A = x^\top A x > 0$$

for every nonzero vector x . Therefore, $\langle \cdot, \cdot \rangle_A$ is an inner product and $\|\cdot\|_A$ is the associated norm. $\square$

1.3 Equivalent quadratic minimization problem

The Conjugate Gradient method can be derived by reformulating the linear system as an optimization problem. Consider the quadratic function

$$f(x) = \frac{1}{2}x^\top A x - b^\top x + c,$$

where $c \in \mathbb{R}$ is a constant.

Proposition 1.2 (Gradient of the quadratic function). *If A is symmetric, then*

$$\nabla f(x) = Ax - b.$$

Proof

The derivative of $\frac{1}{2}x^\top A x$ is Ax when A is symmetric, while the derivative of $-b^\top x$ is $-b$. Therefore,

$$\nabla f(x) = Ax - b.$$

$\square$

Theorem 1.1 (Equivalence between linear system and minimization). *Assume that A is symmetric positive definite. Then solving*

$$Ax = b$$

is equivalent to minimizing

$$f(x) = \frac{1}{2}x^\top A x - b^\top x + c.$$

Proof

A minimizer of f must satisfy the first-order optimality condition

$$\nabla f(x) = 0.$$

Since

$$\nabla f(x) = Ax - b,$$

this condition is equivalent to

$$Ax = b.$$

Because A is positive definite, the Hessian of f is positive definite. Thus f is strictly convex and the minimizer is unique. $\square$

1.4 Residual, error and gradient

It is useful to distinguish the residual, the error and the gradient of the quadratic function.

Definition 1.3 (Residual and error). Let x^* be the exact solution of $Ax = b$. For an approximation x_k , the residual is defined by

$$r_k = b - Ax_k,$$

and the error is defined by

$$e_k = x^* - x_k.$$

Proposition 1.3 (Link between residual and error). *For every iteration k , the residual and the error satisfy*

$$r_k = Ae_k.$$

Moreover, if $g_k = Ax_k - b$ denotes the gradient of the quadratic function, then

$$g_k = -r_k.$$

Proof

Since x^* is the exact solution, we have $Ax^* = b$. Hence,

$$r_k = b - Ax_k = Ax^* - Ax_k = A(x^* - x_k) = Ae_k.$$

Furthermore, the gradient of the quadratic function is

$$g_k = Ax_k - b.$$

Therefore,

$$g_k = -(b - Ax_k) = -r_k.$$

This shows that making the residual small is equivalent to making the gradient close to zero. $\square$

Theorem 1.2 (Strict convexity and uniqueness). *If A is symmetric positive definite, then the function*

$$f(x) = \frac{1}{2}x^\top Ax - b^\top x + c$$

is strictly convex on $\mathbb{R}^n$. Consequently, it admits at most one minimizer.

Proof

The Hessian matrix of f is

$$\nabla^2 f(x) = A.$$

Since A is symmetric positive definite, the Hessian is positive definite for all $x \in \mathbb{R}^n$. Therefore, f is strictly convex. A strictly convex function can have at most one minimizer. Since the first-order condition $\nabla f(x) = 0$ is equivalent to $Ax = b$, the minimizer is unique and coincides with the solution of the linear system. $\square$

Conjugate Gradient Method

2.1 A-conjugacy

Definition 2.1 (A-conjugate vectors). Two vectors p_i and p_j are said to be A-conjugate if

$$p_i^\top A p_j = 0 \quad \text{for } i \neq j.$$

Remark 2.1. A-conjugacy generalizes the usual notion of orthogonality. In the Conjugate Gradient method, search directions are constructed to be mutually A-conjugate. This ensures that minimizing along one direction does not destroy the minimization already achieved along previous directions.

2.2 Exact line search

The step length used in the Conjugate Gradient method is obtained by minimizing the quadratic function along the current search direction.

Proposition 2.1 (Optimal step length). Let d_k be a nonzero search direction. The value of ρ_k that minimizes

$$\varphi(\rho) = f(x_k + \rho d_k)$$

is

$$\rho_k = -\frac{g_k^\top d_k}{d_k^\top A d_k}.$$

Proof

Define

$$\varphi(\rho) = f(x_k + \rho d_k).$$

Using the expression of f , we get

$$\varphi(\rho) = \frac{1}{2}(x_k + \rho d_k)^\top A(x_k + \rho d_k) - b^\top (x_k + \rho d_k) + c.$$

Differentiating with respect to ρ gives

$$\varphi'(\rho) = d_k^\top A(x_k + \rho d_k) - b^\top d_k.$$

Since $g_k = Ax_k - b$, this becomes

$$\varphi'(\rho) = d_k^\top g_k + \rho d_k^\top A d_k.$$

At the minimum, $\varphi'(\rho_k) = 0$, hence

$$d_k^\top g_k + \rho_k d_k^\top A d_k = 0.$$

Therefore,

$$\rho_k = -\frac{d_k^\top g_k}{d_k^\top A d_k} = -\frac{g_k^\top d_k}{d_k^\top A d_k}.$$

Since A is positive definite and $d_k \neq 0$, we have $d_k^\top A d_k > 0$, so this value is well-defined. $\square$

2.3 Algorithm

The Fletcher-Reeves version of the Conjugate Gradient method is initialized by choosing $x_0 \in \mathbb{R}^n$ and setting

$$g_0 = Ax_0 - b, \quad d_0 = -g_0.$$

At iteration k , the step length is

$$\rho_k = -\frac{g_k^\top d_k}{d_k^\top A d_k}.$$

Then

$$x_{k+1} = x_k + \rho_k d_k,$$

and the new gradient is

$$g_{k+1} = Ax_{k+1} - b.$$

The new direction is defined by

$$d_{k+1} = -g_{k+1} + \beta_k d_k,$$

where

$$\beta_k = \frac{\|g_{k+1}\|^2}{\|g_k\|^2}.$$

Algorithm 1 Conjugate Gradient Method

- 1: Choose $x_0 \in \mathbb{R}^n$
 - 2: $g_0 \leftarrow Ax_0 - b$
 - 3: $d_0 \leftarrow -g_0$
 - 4: $k \leftarrow 0$
 - 5: **while** $\|g_k\| > \text{tol}$ and $k < \text{max_iter}$ **do**
 - 6: $\rho_k \leftarrow -\frac{g_k^\top d_k}{d_k^\top A d_k}$
 - 7: $x_{k+1} \leftarrow x_k + \rho_k d_k$
 - 8: $g_{k+1} \leftarrow Ax_{k+1} - b$
 - 9: $\beta_k \leftarrow \frac{\|g_{k+1}\|^2}{\|g_k\|^2}$
 - 10: $d_{k+1} \leftarrow -g_{k+1} + \beta_k d_k$
 - 11: $k \leftarrow k + 1$
 - 12: **end while**
 - 13: **return** x_k
-

2.4 Interpretation of the steps

The algorithm can be understood as follows:

- g_k is the gradient of the quadratic objective at x_k ;
- d_k is the search direction;
- ρ_k is the exact step length minimizing f along d_k ;
- β_k ensures that the new direction remains conjugate to the previous ones.

Orthogonality and Finite Convergence

3.1 Orthogonality of gradients

The Conjugate Gradient method constructs a sequence of gradients (g_k) that are mutually orthogonal:

$$g_i^\top g_j = 0 \quad \text{for } i \neq j.$$

Remark 3.1. This property explains why the method can terminate in a finite number of steps in exact arithmetic. Since it is impossible to have more than n nonzero mutually orthogonal vectors in $\mathbb{R}^n$, the gradient must become zero after at most n iterations.

3.2 Krylov subspaces

Definition 3.1 (Krylov subspace). The k -th Krylov subspace associated with A and g_0 is defined by

$$\mathcal{K}_k(A, g_0) = \text{span}\{g_0, Ag_0, A^2g_0, \dots, A^{k-1}g_0\}.$$

The Conjugate Gradient iterates satisfy

$$x_k \in x_0 + \mathcal{K}_k(A, g_0).$$

Lemma 3.1. Let $U_k = \mathcal{K}_{k+1}(A, g_0)$. The sequence generated by the Conjugate Gradient method satisfies:

1. $x_k \in x_0 + U_{k-1}$;
2. $g_k \in U_k$;
3. $g_{k+1} = g_k + As_k$, where $s_k = x_{k+1} - x_k$;
4. $s_k \in U_k$.

Proof

Since each iterate is obtained by adding a search direction belonging to a Krylov subspace, we have $x_k \in x_0 + U_{k-1}$. Moreover,

$$g_k = Ax_k - b.$$

Using the expression of x_k as an element of $x_0 + U_{k-1}$ shows that $g_k \in U_k$. Finally,

$$g_{k+1} = Ax_{k+1} - b = A(x_k + s_k) - b = g_k + As_k.$$

Since both x_{k+1} and x_k belong to affine spaces generated by Krylov subspaces, their difference s_k belongs to U_k . $\square$

Theorem 3.1 (Finite convergence of the Conjugate Gradient method). *Let $A \in \mathbb{R}^{n \times n}$ be symmetric positive definite. In exact arithmetic, the Conjugate Gradient method converges to the exact solution of*

$$Ax = b$$

in at most n iterations.

Proof

The Conjugate Gradient method constructs mutually orthogonal gradients and mutually A-conjugate search directions. Since $\mathbb{R}^n$ cannot contain more than n nonzero mutually orthogonal vectors, the gradient must become zero after at most n iterations. When $g_k = 0$, we have

$$Ax_k - b = 0,$$

and therefore

$$Ax_k = b.$$

Thus x_k is the exact solution. $\square$

Detailed Numerical Example

4.1 Problem setup

Consider the system

$$Ax = b$$

with

$$A = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad x_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

4.2 Initialization

The initial gradient is

$$g_0 = Ax_0 - b = \begin{pmatrix} -1 \\ -2 \\ -3 \end{pmatrix},$$

and the first search direction is

$$d_0 = -g_0 = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}.$$

4.3 First iteration

We compute

$$Ad_0 = \begin{pmatrix} 6 \\ 10 \\ 8 \end{pmatrix}, \quad d_0^\top Ad_0 = 50, \quad g_0^\top d_0 = -14.$$

Therefore,

$$\rho_0 = -\frac{g_0^\top d_0}{d_0^\top Ad_0} = -\frac{-14}{50} = 0.28.$$

Hence,

$$x_1 = x_0 + \rho_0 d_0 = \begin{pmatrix} 0.28 \\ 0.56 \\ 0.84 \end{pmatrix}.$$

The new gradient is

$$g_1 = Ax_1 - b = \begin{pmatrix} -0.68 \\ -0.80 \\ 0.76 \end{pmatrix}.$$

The Fletcher-Reeves coefficient is

$$\beta_0 = \frac{\|g_1\|^2}{\|g_0\|^2} = \frac{1.68}{14} = 0.12.$$

Thus,

$$d_1 = -g_1 + \beta_0 d_0 = \begin{pmatrix} 0.80 \\ 1.04 \\ -0.40 \end{pmatrix}.$$

4.4 Second and third iterations

Continuing the same computations gives

$$x_2 \approx \begin{pmatrix} 0.1467 \\ 0.3867 \\ 0.9067 \end{pmatrix}, \quad g_2 \approx \begin{pmatrix} -0.0733 \\ -0.2267 \\ -0.0667 \end{pmatrix}.$$

After the third iteration, the exact solution is obtained:

$$x_3 = \begin{pmatrix} -0.1250 \\ 1.5000 \\ 0.7500 \end{pmatrix}, \quad g_3 \approx \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

4.5 Verification properties

The directions are A-conjugate:

$$d_0^\top A d_1 = 0, \quad d_1^\top A d_2 \approx 0.$$

The gradients are orthogonal:

$$g_0^\top g_1 = 0, \quad g_1^\top g_2 \approx 0, \quad g_0^\top g_2 \approx 0.$$

4.6 Numerical verification with Python

To complete the theoretical computations, we implemented the Conjugate Gradient method in Python. The objective of the implementation is to verify that the algorithm returns the same solution as the one obtained by manual computation.

For the matrix

$$A = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix},$$

the Python implementation returns the solution

$$x = \begin{pmatrix} -0.125 \\ 1.5 \\ 0.75 \end{pmatrix}.$$

This confirms the result obtained in the detailed numerical example. In addition, the residual norm becomes close to zero after three iterations, which is consistent with the finite convergence property of the Conjugate Gradient method in exact arithmetic for a three-dimensional symmetric positive definite system.

Applications and Practical Considerations

5.1 Geometric interpretation

For a three-dimensional problem, the Conjugate Gradient method can be interpreted geometrically:

- at the first iteration, the method searches along a line;
- at the second iteration, it searches in a plane;
- at the third iteration, it searches in the full space $\mathbb{R}^3$.

Iteration	Search space
0	Initial point x_0
1	Line $x_0 + \text{span}\{g_0\}$
2	Plane $x_0 + \text{span}\{g_0, Ag_0\}$
3	Full space $\mathbb{R}^3$

5.2 Preconditioning

In practice, the convergence of the Conjugate Gradient method can be slow when the matrix A is ill-conditioned. A common improvement is preconditioning. Instead of solving

$$Ax = b,$$

one solves a transformed system such as

$$M^{-1}Ax = M^{-1}b,$$

where M is chosen as an approximation of A that is easier to invert.

5.3 Implementation strategy

The numerical implementation of the Conjugate Gradient method follows the theoretical algorithm presented in Chapter 2. The method is particularly efficient because it does not require the explicit computation of a matrix inverse. Instead, each iteration only relies on matrix-vector products, scalar products and vector updates.

This is an important computational advantage. For large sparse matrices, storing the full matrix or computing a direct factorization may be expensive. In contrast, the Conjugate Gradient method can exploit sparsity and requires only a limited amount of memory. This makes it suitable for applications in numerical optimization, scientific computing and large-scale linear systems.

In the implementation, the stopping criterion is based on the Euclidean norm of the residual

$$r_k = b - Ax_k.$$

Since the gradient of the quadratic objective is given by

$$g_k = Ax_k - b = -r_k,$$

checking whether the residual norm is small is equivalent to checking whether the gradient is close to zero. When

$$\|r_k\| < \text{tol},$$

the algorithm stops and returns the current approximation x_k .

The implementation also stores the history of the residual norms. This allows us to analyze the convergence of the algorithm and verify that the residual decreases during the iterations.

5.4 Stopping criteria and numerical stability

In exact arithmetic, the Conjugate Gradient method converges in at most n iterations for a matrix $A \in \mathbb{R}^{n \times n}$ that is symmetric positive definite. However, in floating-point arithmetic, round-off errors may affect the orthogonality of the gradients and the A -conjugacy of the directions.

For this reason, practical implementations do not usually wait for the residual to become exactly zero. Instead, they use a stopping criterion such as

$$\frac{\|r_k\|}{\|b\|} < \varepsilon,$$

where ε is a prescribed tolerance. This relative criterion is often preferable to an absolute criterion because it takes into account the scale of the right-hand side vector b .

Remark 5.1. A very small tolerance does not always improve the quality of the solution in finite precision. After a certain number of iterations, round-off errors may dominate and the residual norm may stagnate. Therefore, the tolerance must be chosen according to the numerical precision and the required accuracy of the application.

5.5 Computational cost

At each iteration, the main cost of the Conjugate Gradient method is the matrix-vector product Ad_k . If A is dense, this operation costs approximately $O(n^2)$ operations. If A is sparse, the cost is proportional to the number of nonzero coefficients of A , which can be much smaller.

The memory cost is also limited. The algorithm only needs to store a few vectors of size n , such as x_k , r_k and d_k . This gives a memory complexity of order $O(n)$, excluding the storage of the matrix itself.

Operation	Role in the algorithm
Matrix-vector product Ad_k	Main computational cost of one iteration
Scalar products	Used to compute the step size and the update coefficient
Vector updates	Used to update the approximation, residual and direction
Residual norm	Used to monitor convergence and stop the algorithm

5.6 Preconditioned Conjugate Gradient method

The convergence of the Conjugate Gradient method depends strongly on the condition number $\kappa(A)$. When $\kappa(A)$ is large, the level sets of the quadratic function are elongated ellipsoids and the convergence may become slow. A standard remedy is to introduce a preconditioner.

Let M be a symmetric positive definite matrix that approximates A and is easier to invert. The preconditioned system can be written as

$$M^{-1}Ax = M^{-1}b.$$

The objective is to choose M such that the transformed system has a smaller condition number than the original one.

Definition 5.1 (Preconditioner). A preconditioner is a matrix M chosen to improve the numerical properties of a linear system. In the context of the Conjugate Gradient method, M should be symmetric positive definite and close enough to A , while remaining easy to solve in systems of the form $Mz = r$.

Remark 5.2. A good preconditioner reduces the number of iterations, but each iteration becomes slightly more expensive because one must solve an auxiliary system involving M . Therefore, the efficiency of preconditioning depends on the balance between the reduction in iterations and the additional cost per iteration.

5.7 Practical workflow

A complete numerical workflow for applying the Conjugate Gradient method can be summarized as follows:

1. Check that the matrix A is symmetric.
2. Verify that A is positive definite, for example through Cholesky factorization or eigenvalue analysis.
3. Choose an initial point x_0 .
4. Run the Conjugate Gradient algorithm with a tolerance adapted to the problem.
5. Monitor the residual norm $\|r_k\|$.

6. Compare the result with a direct solver when the dimension is small enough.
7. For large or ill-conditioned systems, consider preconditioning.

5.8 Possible extensions

Several extensions could be considered to make this project more complete. First, one could compare the Conjugate Gradient method with the steepest descent method on the same quadratic function. This would highlight the advantage of conjugate directions over purely gradient-based descent directions.

Second, one could study the impact of the condition number by generating symmetric positive definite matrices with different eigenvalue distributions. This would make it possible to visualize how conditioning affects the residual norm and the number of iterations.

Third, one could implement a preconditioned version of the algorithm and compare the convergence with and without preconditioning. This would connect the theoretical convergence bound with practical numerical performance.

5.9 Advantages and limitations

Advantages	Limitations
Finite convergence in exact arithmetic	Requires a symmetric positive definite matrix
Low memory cost	Sensitive to conditioning
Well suited for large sparse systems	May require preconditioning
Strong theoretical guarantees	Loss of orthogonality may occur numerically

Conclusion

The Conjugate Gradient method is an elegant and powerful algorithm that connects numerical linear algebra with optimization theory. By reformulating the linear system $Ax = b$ as the minimization of a strictly convex quadratic function, the method constructs a sequence of iterates using gradients and conjugate search directions.

Its main theoretical strength lies in its finite convergence property. In exact arithmetic, for a symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$, the method reaches the exact solution in at most n iterations. This result is explained by the orthogonality of gradients, the A-conjugacy of search directions and the structure of Krylov subspaces.

From a practical point of view, the method is especially useful for large sparse systems because it requires only matrix-vector products and has a low memory footprint. However, its efficiency may deteriorate for ill-conditioned matrices, which motivates the use of preconditioning.

Overall, the Conjugate Gradient method remains one of the most important iterative methods in computational mathematics, scientific computing and numerical optimization.

This project also shows the importance of combining mathematical reasoning with computational verification. The theoretical results explain why the method works, while the Python implementation makes it possible to observe the behavior of the residuals, validate the numerical solution, and understand the practical role of stopping criteria and conditioning.

References

1. J. R. Shewchuk, *An Introduction to the Conjugate Gradient Method Without the Agonizing Pain*, Carnegie Mellon University, 1994.
2. Y. Saad, *Iterative Methods for Sparse Linear Systems*, SIAM, 2003.
3. J. Nocedal and S. J. Wright, *Numerical Optimization*, Springer, 2006.
4. G. H. Golub and C. F. Van Loan, *Matrix Computations*, Johns Hopkins University Press, 2013.

Appendix: Python Code

This appendix contains the Python implementation used to reproduce the numerical example presented in Chapter 4. The code implements the Conjugate Gradient algorithm from scratch, checks the assumptions on the matrix, prints the intermediate iterations, and compares the final result with NumPy's direct linear solver.

```
1 import numpy as np
2
3
4 def is_symmetric_positive_definite(A, tol=1e-12):
5     """
6     Check whether a matrix is symmetric positive definite.
7
8     A matrix A is symmetric positive definite if:
9     1. A is symmetric: A.T = A
10    2. all eigenvalues of A are strictly positive
11    """
12    if not np.allclose(A, A.T, atol=tol):
13        return False
14
15    eigenvalues = np.linalg.eigvals(A)
16    return np.all(eigenvalues > tol)
17
18
19 def quadratic_function(A, b, x):
20     """
21     Compute the quadratic objective associated with Ax = b:
22
23      $f(x) = 1/2 x.T A x - b.T x$ 
24     """
25     return 0.5 * x.T @ A @ x - b.T @ x
26
27
28 def conjugate_gradient(A, b, x0=None, tol=1e-10, max_iter=None,
29                       verbose=True):
30     """
31     Solve the linear system Ax = b using the Conjugate Gradient method.
32     """
33     n = len(b)
34
35     if x0 is None:
36         x = np.zeros(n)
37     else:
38         x = x0.astype(float).copy()
39
40     if max_iter is None:
41         max_iter = n
42
43     # Residual  $r_k = b - A x_k$ .
44     # Since  $g_k = A x_k - b = -r_k$ , the first descent direction is  $r_0$ .
45     r = b - A @ x
46     d = r.copy()
47
48     history = [np.linalg.norm(r)]
49
50     if verbose:
51         print("Initial point:")
```

```

52     print("x0 =", x)
53     print("Initial residual norm =", history[-1])
54     print("-" * 60)
55
56     for k in range(max_iter):
57         Ad = A @ d
58         denominator = d.T @ Ad
59
60         if abs(denominator) < 1e-15:
61             raise ValueError("The denominator is too close to zero.")
62
63         alpha = (r.T @ r) / denominator
64         x_new = x + alpha * d
65         r_new = r - alpha * Ad
66
67         residual_norm = np.linalg.norm(r_new)
68         history.append(residual_norm)
69
70         if verbose:
71             print(f"Iteration {k + 1}")
72             print("alpha =", alpha)
73             print("x =", x_new)
74             print("residual =", r_new)
75             print("residual norm =", residual_norm)
76             print("objective value =", quadratic_function(A, b, x_new))
77             print("-" * 60)
78
79         if residual_norm < tol:
80             return x_new, history
81
82         beta = (r_new.T @ r_new) / (r.T @ r)
83         d = r_new + beta * d
84
85         x = x_new
86         r = r_new
87
88     return x, history
89
90
91 # Numerical example from the report
92 A = np.array([
93     [4.0, 1.0, 0.0],
94     [1.0, 3.0, 1.0],
95     [0.0, 1.0, 2.0]
96 ])
97
98 b = np.array([1.0, 2.0, 3.0])
99 x0 = np.array([0.0, 0.0, 0.0])
100
101 print("Is A symmetric positive definite?", is_symmetric_positive_definite(A))
102 print("-" * 60)
103
104 x_cg, residual_history = conjugate_gradient(
105     A=A,
106     b=b,
107     x0=x0,
108     tol=1e-12,
109     max_iter=10,
110     verbose=True
111 )
112
113 x_direct = np.linalg.solve(A, b)
114
115 print("Final solution obtained with Conjugate Gradient:")
116 print(x_cg)
117
118 print("Direct solution using numpy.linalg.solve:")

```

```

119 print(x_direct)
120
121 print("Difference between both solutions:")
122 print(np.linalg.norm(x_cg - x_direct))
123
124 print("Residual history:")
125 print(residual_history)
126
127 print_convergence_table(residual_history)

```

Listing 1: Conjugate Gradient Python implementation

The output confirms that the matrix is symmetric positive definite and that the Conjugate Gradient method reaches the exact solution in three iterations for this three-dimensional example. The solution is

$$x = \begin{pmatrix} -0.125 \\ 1.5 \\ 0.75 \end{pmatrix},$$

which is consistent with the theoretical computations carried out in Chapter 4.

```

1 Is A symmetric positive definite? True
2 =====
3 Initial point:
4 x0 = [0. 0. 0.]
5 Initial residual norm = 3.7416573867739413
6 -----
7 Iteration 1
8 alpha = 0.28
9 x = [0.28 0.56 0.84]
10 residual = [ 0.68  0.8 -0.76]
11 residual norm = 1.296148139681572
12 objective value = -1.96
13 -----
14 Iteration 2
15 x approximately = [0.14666667 0.38666667 0.90666667]
16 -----
17 Iteration 3
18 x approximately = [-0.125  1.5    0.75]
19 residual norm approximately = 0
20 -----
21 Final solution obtained with Conjugate Gradient:
22 [-0.125  1.5    0.75]

```

Listing 2: Expected output of the Python implementation